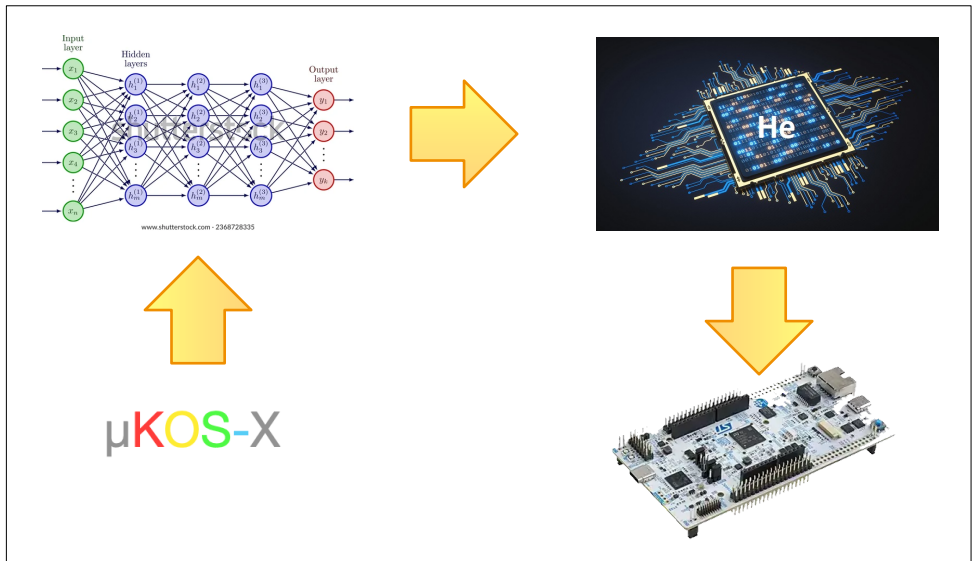


v 1.3

mlpn, NN library performances

uKOS-X: © 2025-2026, Edo. Franzì



Introduction

µKOS-X embeds an **optimised Multi-Layer Perceptron (MLP) fully connected neural network library**, specifically designed for deeply embedded systems with strict performance, memory, and power constraints. The library has been engineered to maximise computational efficiency on modern microcontroller architectures while maintaining a compact footprint and high portability across platforms.

The primary objective of this white paper is to **highlight, analyse, and benchmark the substantial performance gains** enabled by recent vector-processing hardware innovations, such as **Arm Helium** (M-Profile Vector Extension, **MVE**). These SIMD vector extensions significantly accelerate the core mathematical kernels involved in neural network inference, including multiply-accumulate operations, vector reductions, and activation functions, which traditionally dominate execution time on embedded targets.

To provide a concrete and reproducible evaluation, the study focuses on a **deliberately simple classification use case**, representative of many real-world industrial applications such as signal classification, anomaly detection, or lightweight pattern recognition. The classifier is implemented using a compact MLP architecture and executed on the **ST Nucleo-N657** reference platform, offering a realistic embedded execution environment without relying on external accelerators.

Multiple implementations are evaluated and compared, including:

- I. µKOS-X mlpn library without Helium
- II. µKOS-X mlpn library with Helium
- III. TensorFlow Lite with Helium

The benchmarking results assess **execution latency, CPU utilisation, and memory footprint**, clearly demonstrating the advantages of vectorised processing for embedded machine learning workloads. Particular attention is given to performance-per-cycle and scalability, providing insight into how modern MCU architectures can effectively support edge AI applications.

Source code, benchmark configurations, and build scripts are provided as part of the **µKOS-X package**, ensuring full transparency and reproducibility of the results presented. This white paper is intended for system architects, embedded software developers, and AI engineers seeking to **deploy efficient on-device inference capabilities** on resource-constrained platforms, without the overhead of complex operating systems or dedicated hardware accelerators.

Considered example

For this example, a **simple three-layer Multi-Layer Perceptron (MLP)** architecture is considered. The network processes **two input features**, followed by two hidden layers: **Layer L1 comprising 52 neurones** and **Layer L2 comprising 73 neurones**. The final layer (**L3**) contains **three output neurones**, which encode the target class using a compact representation suitable for binary classification.

The network employs the **arctangent (atan) activation function** across its layers. This choice provides a smooth, bounded, and differentiable nonlinearity, which is particularly well suited for embedded inference due to its numerical stability and predictable dynamic range. Although more complex activation functions exist, **atan** offers a favorable trade-off between computational cost and classification robustness in constrained environments.

This network topology has been deliberately selected to strike a balance between **computational complexity and representativeness**. While remaining sufficiently lightweight to execute on a microcontroller-class device, it is large enough to meaningfully stress the core mathematical kernels —namely dense matrix–vector multiplications and activation evaluations—that benefit most from vectorised execution using modern SIMD extensions.

All corresponding **source files, configuration headers, and build scripts** required to reproduce this example are provided within the µKOS-X distribution. The sources are located in:

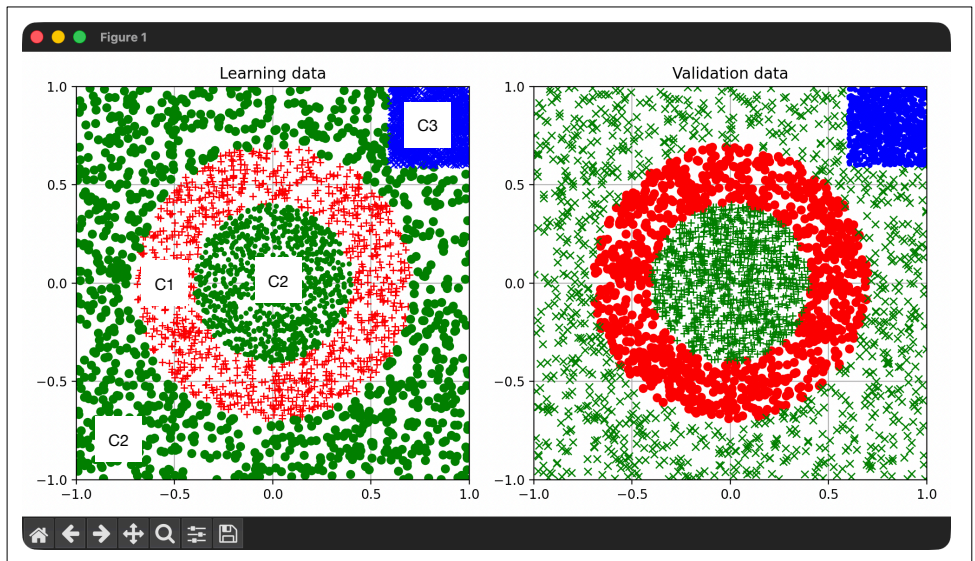
```
uKOS-X/Applications/uKOS_Appls_Downloadable/l_MLPs/class_Py    using the mlpn library
uKOS-X/Applications/uKOS_Appls_Downloadable/l_MLPs/class_TFL    using TensorFlow Lite
```

Package / hardware	Version
uKOS-X	v. 0.1.233
Network number representation	float32_t
gcc	15.2.0
Python	3.14
TensorFlow Lite	hash e028871
Nucleo-N657	ST STM32N657 @ 800-MHz
Nucleo-H753	ST STM32H753 @ 480-MHz

In this example, the neural network model is **fed with randomly generated two-dimensional input vectors (X, Y)**. These inputs are uniformly distributed over the input space and serve to evaluate the classifier's behaviour across the full decision domain rather than on a limited set of predefined test points.

The model produces a **continuous output value representing the detected class**. An output value close to **-1.0 indicates an incorrect or negative class**, whereas a value close to **+1.0 corresponds to the positive (detected) class**. This bipolar output encoding simplifies the decision logic and is well suited for lightweight embedded inference pipelines.

The classification problem addressed by this example involves **Three distinct classes**:



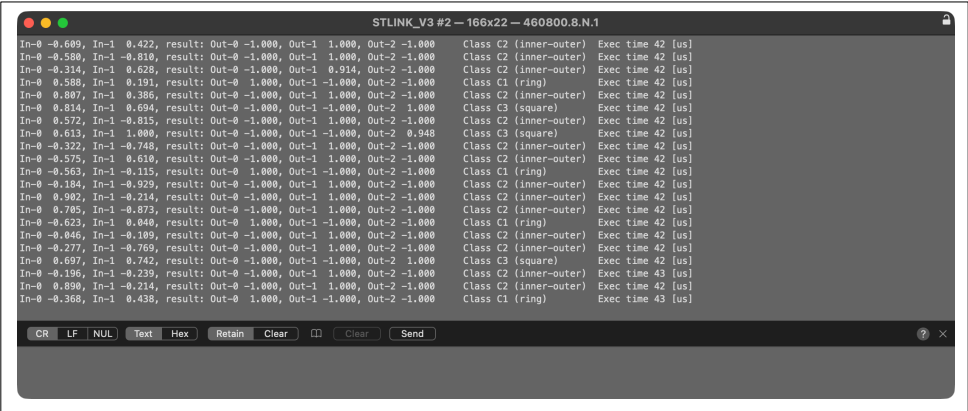
- I. the **red class**, C1, corresponding to a *ring-shaped region* in the input space,
- II. the **green class**, C2, representing the inner and outer regions surrounding the ring.
- III. the **blue class**, C3, representing the square on the top-right.

This deliberately non-linear class topology has been selected to demonstrate the ability of the MLP to learn and represent **non-trivial decision boundaries**, which are common in real-world signal and pattern classification problems.

When an input sample lies **too close to the boundary separating the three classes**, the network output may fall within an intermediate range that does not clearly map to either class. In such cases,

the sample is flagged as **“not well classified”**, highlighting regions of uncertainty in the model’s decision surface. This behaviour provides valuable insight into the classifier’s confidence and illustrates the smooth transition characteristics induced by the chosen activation function.

This test scenario not only validates the functional correctness of the neural network implementation but also serves as an effective workload for evaluating **execution performance, numerical stability, and vectorisation efficiency** on the target embedded platform.



Used library	CPU	Execution time	Size code	Size data
uKOS-X (no Helium)	N657 @ 800-MHz	45-µs	(18328 + 1688)-Bytes	1104-Bytes
uKOS-X (Helium)	N657 @ 800-MHz	24-µs	(18384 + 1680)-Bytes	1104-Bytes
TensorFlow Lite (Helium)	N657 @ 800-MHz	73-µs	62844-Bytes	10260-Bytes
uKOS-X (no Helium)	H753 @ 480-MHz	56-µs	(18556 + 1028)-Bytes	1104-Bytes
TensorFlow Lite (no Helium)	H753 @ 480-MHz	133-µs	66456-Bytes	10260-Bytes

N657 / H753 Clock Ratio = 1.66

H753 Execution time scaled @ the same N657 clock = 56-µs / 1.66 = 33.7-µs

without Helium, the H753 is faster than the N657 for the same clock.

In red the size of the test application

In blue the size of the embedded mlpn library

General Philosophy

μKOS-X

μKOS-X follows a **low-level, deterministic, and hardware-aware design philosophy**. It targets a well-defined class of neural networks—primarily fully connected MLPs—and avoids generic graph-based execution models. The library is explicitly optimised for embedded targets and leverages modern microcontroller hardware features such as SIMD vector units in a controlled and predictable manner.

The focus of μKOS-X is not only on flexibility, but on **maximum efficiency, minimal overhead, and full control over execution behaviour**, making it particularly suitable for industrial and real-time embedded applications.

TensorFlow Lite Micro + CMSIS-NN

TensorFlow Lite Micro adopts a **generic and portable inference model**, designed to execute TensorFlow computational graphs on resource-constrained devices. **CMSIS-NN** is used as an optional optimised backend to accelerate selected operators on Arm Cortex-M devices.

This approach prioritises ecosystem compatibility, model portability, and development flexibility, at the cost of increased runtime complexity and abstraction layers.

Use of Arm Helium (MVE)

μKOS-X

μKOS-X makes **direct and explicit use** of the **Arm Helium** (MVE) vector unit. Vectorisation is deliberately designed into the core kernels, ensuring:

- I. predictable SIMD width usage,
- II. controlled memory alignment,
- III. stable and repeatable code generation.

This results in **optimal and consistent exploitation of Helium**, with minimal overhead and no dependency on external kernel selection mechanisms.

TensorFlow Lite Micro + CMSIS-NN

Helium usage is **indirect**, entirely dependent on CMSIS-NN optimised kernels. Whether Helium is actually used depends on:

- I. the availability of a CMSIS-NN implementation for a given operator,
- II. the data type and quantisation scheme,
- III. the structure of the TensorFlow graph.

As a result, Helium utilisation is **opportunistic rather than guaranteed**, and may vary significantly between models.

Raw Performance (Latency and Cycle Count)

μKOS-X

μKOS-X avoids graph traversal, dynamic dispatch, and operator resolution. Neural network inference is implemented as a sequence of **direct function calls**, with no intermediate layers.

This leads to:

- I. minimal instruction overhead,
- II. reduced branching,
- III. cycle-accurate and highly predictable execution.

μKOS-X therefore delivers **very low inference latency** and excellent performance-per-cycle.

TensorFlow Lite Micro + CMSIS-NN

Even when using optimised kernels, TensorFlow Lite Micro introduces **structural overhead** due to:

- I. graph interpretation,
- II. operator dispatch,
- III. tensor bookkeeping.

This overhead is intrinsic to the runtime architecture and cannot be fully eliminated, resulting in **higher latency compared to a specialised implementation**.

Memory Footprint (FLASH and RAM)

μKOS-X

μKOS-X has a **very small memory footprint**, as it includes only the code strictly required by the application:

- I. no generic runtime,
- II. no tensor arena,
- III. no operator tables.

Typical deployments require only a few kilobytes to a few tens of kilobytes of FLASH, with minimal RAM usage.

TensorFlow Lite Micro + CMSIS-NN

TensorFlow Lite Micro requires:

- I. the runtime core,
- II. operator resolver tables,
- III. a tensor arena for intermediate buffers,
- IV. graph metadata.

This leads to a **significantly larger FLASH and RAM footprint**, often several times larger than a specialised solution.

Real-Time Determinism

μKOS-X

μKOS-X is designed for fully deterministic execution:

- I. no dynamic memory allocation,
- II. fixed execution paths,
- III. constant-time inference.

This makes it well suited for hard real-time and safety-critical systems.

TensorFlow Lite Micro + CMSIS-NN

Although TensorFlow Lite Micro typically uses static memory allocation, execution behaviour still depends on:

- I. graph structure,
- II. operator sequence,
- III. model-specific optimisations.

As a result, strict real-time determinism is harder to guarantee.

Embedded Integration and Debugging

µKOS-X

Integration into an embedded firmware is straightforward:

- I. simple C APIs,
- II. no external tooling required at runtime,
- III. clear call stacks and predictable control flow.

Debugging and profiling are therefore **simple and efficient**, using standard embedded tools.

TensorFlow Lite Micro + CMSIS-NN

The integration flow is more complex:

- I. model conversion and validation,
- II. operator compatibility constraints,
- III. runtime configuration.

Debugging inference issues often requires navigating multiple abstraction layers, increasing development complexity.

Model Flexibility and Scalability

μKOS-X

μKOS-X targets **fixed and well-defined network architectures**. Changes to the network topology typically require recompilation.

This approach is ideal for **production systems with stable requirements**, but less suitable for rapid model iteration.

TensorFlow Lite Micro + CMSIS-NN

TensorFlow Lite Micro supports a wide variety of model architectures. Models can often be changed without modifying application code, making it highly suitable for **research, prototyping, and experimentation**.

Positioning Statement

μKOS-X is a purpose-built embedded inference solution designed to extract maximum performance from modern microcontroller architectures, particularly those featuring **Arm Helium vector extensions**. By avoiding the overhead of generic ML runtimes and graph-based execution, μKOS-X delivers superior determinism, reduced memory footprint, and predictable real-time behaviour.

In contrast, **TensorFlow Lite Micro** combined with **CMSIS-NN** prioritises portability and ecosystem compatibility, offering greater model flexibility at the cost of increased runtime overhead and less controlled utilisation of vector hardware resources.

Criterion	μKOS-X	TensorFlow Lite Micro + CMSIS-NN
Design philosophy	Specialised, low-level inference library focused on fully connected MLP networks	Generic embedded ML runtime executing TensorFlow graphs
Target use cases	Fixed-function embedded AI, industrial products, real-time systems	Prototyping, portability, TensorFlow ecosystem integration
Execution model	Direct function calls, no graph, no dispatcher	Graph-based execution with operator resolution
Helium (MVE) usage	Helium (MVE) usage	Helium (MVE) usage
Vectorization control	Fully controlled and predictable SIMD usage	Opportunistic, depends on kernel availability and graph structure
Performance (latency / cycles)	Minimal overhead, cycle-accurate and deterministic	Higher overhead due to runtime and graph traversal
FLASH footprint	Very low (typically a few kB to tens of kB)	Medium to high (runtime, operators, metadata)
RAM footprint	Minimal, no generic tensor arena	Requires tensor arena and intermediate buffers
Determinism	Fully deterministic, no dynamic behavior	Partially deterministic, graph-dependent execution paths
Real-time suitability	Excellent (hard real-time compatible)	Limited (soft real-time at best)
Memory allocation	Static, application-controlled	Static arena but runtime-managed
Debugging and traceability	Simple (standard C functions, clear call stack)	More complex (graph, operators, runtime layers)
Model flexibility	Fixed architecture, recompile required for changes	High flexibility, models can be swapped without code changes
Supported network types	Optimised MLP (fully connected, other models in preparation)	Wide range (CNN, FC, RNN, etc., depending on support)
Dependency footprint	Minimal, self-contained	Heavy (TensorFlow Lite Micro + CMSIS + conversion tools)
Industrial deployment readiness	High	Moderate
Best suited for	Performance-critical, resource-constrained embedded products	Rapid development, experimentation, ML research